Generalized Reserve Set Covering Problem with Buffer and Connectivity Requirements

Camilla Bigotto, Anna Guccione

UNIVERSITÀ
DEGLI STUDI
DI TRIESTE

Introduction

The GRSC-CB problem

DEFINITION OF THE PROBLEM

- Design of nature reserves for ensuring the conservation of endangered wildlife
- Reserves must verify a series of spatial requirements:
  - **Connectivity**: to avoid spatial fragmentation
  - **Buffer zones**: surrounding/protecting the core areas
- **General Reserve Set Covering Problem with Connectivity and Buffer Requirements**

The GRSC-CB problem

SOLUTION

- Introduced in a modular way
  - RSC → GRSC → GRSC-B → GRSC-C → GRSC-CB
- Solution based on Integer Linear Programming and **branch-and-cut**
- Enhanced model by:
  - Valid inequalities
  - Construction and primal heuristic
  - Local branching heuristic

Generalized Reserve Set Covering Problem

RSC: Reserve Set Covering problem

DEFINITIONS

- Given:
  - A set V of **land sites**
  - A set of **species** S
  - $\forall s \in S$ a set of suitable land sites $V_s \subseteq V$
- Find the minimum number of reserve sites such that each species is presented in the selected set of sites at least once.

RSC: Reserve Set Covering problem

VARIABLE

x_i with $1 \leq i \leq |V|$ defined as:

$$x_i = \begin{cases} 1, & \text{if the land site } i \in V \text{ is selected} \\ 0, & \text{otherwise} \end{cases}$$

CONSTRAINT

COV

Each specie needs to be covered by a suitable land site.

$$\sum_{i \in V_s} x_i \geq 1, \forall s \in S$$

OBJECTIVE FUNCTION

Minimize the number of land sites

$$\min \pi(x) = \sum_{i \in V} x_i$$

GRSC: Generalized RSC

DEFINITIONS

Quotas of ecological suitability

- $S_1 \subseteq S$ → set of species that need to stay in the core area
- $S_2 \subseteq S, S_1 \cup S_2 = S, S_1 \cap S_2 = \varnothing$ → the other species
- $w : V \times S \to \mathbb{R}^+$ → **habitat suitability function** measures how advisable is a site $i \in V$ for a specie $s \in S$
- therefore we can define $V_s = \{i \in V | w(s, i) > 0\}$
- $\lambda_s \geq 0$ → **minimum quota of ecological suitability** for s

GRSC: Generalized RSC

DEFINITIONS

Protection

- $s \in S$ is considered **protected** if $\sum_i w(i, s) \geq \lambda_s$
- $0 \leq P_1 \leq |S_1|$, $0 \leq P_2 \leq |S_2|$, minimum number of species respectively in S_1 and S_2 that the reserve needs to protect

Cost

- $c : V \to \mathbb{R}^+$ → **cost function** of choosing the site $i \in V$ to be in the reserve

GRSC: Generalized RSC

VARIABLES

- x_i defined as before
- z_i, $1 \leq i \leq |V|$ → binary variables defined as:

$$z_i = \begin{cases} 1, & \text{if the land site } i \in V \text{ is part of the core area} \\ 0, & \text{otherwise} \end{cases}$$

- u_s, $1 \leq s \leq |S|$ → binary variables defined as:

$$u_s = \begin{cases} 1, & \text{if the specie } s \in S \text{ is protected by the reserve} \\ 0, & \text{otherwise} \end{cases}$$

The solution will be a tuple (u, x, z)

GRSC: Generalized RSC

CONSTRAINTS

S1-SQ

If a specie in S_1 is protected, the habitat suitability score for the land sites part of the core area must be at least the minimum quota of ecological suitability of that specie.

$$\sum_{i \in V_s} w(i,s) z_i \geq \lambda_s u_s \quad \forall s \in S_1$$

S2-SQ

Similarly for a specie in S_2, considering land sites in the whole reserve.

$$\sum_{i \in V_s} w(i,s) x_i \geq \lambda_s u_s \quad \forall s \in S_2$$

GRSC: Generalized RSC

CONSTRAINTS

S1-PROTECT

At least P_1 species of S_1 must be protected.

$$\sum_{s \in S_1} u_s \geq P_1$$

S2-PROTECT

At least P_2 species of S_2 must be protected.

$$\sum_{s \in S_2} u_s \geq P_2$$

GRSC: Generalized RSC

CONSTRAINTS

LINK

If a site is in the core, then it must also be in the reserve.

$$z_i \leq x_i \quad \forall i \in V$$

OBJECTIVE FUNCTION

COST

Function that indicates the cost that needs to be **minimized**.

$$\gamma(u, x, z) = \sum_{i \in V} c_i x_i$$

GRSC: Generalized RSC

GRSC MODEL

$$GRSC : \min\{\gamma(u, x, z) | (S1\text{-}SQ), (S2\text{-}SQ), (S1\text{-}PROTECT), (S2\text{-}PROTECT), (LINK)\}$$

Buffer and Connectivity requirements

GRSC-B: GRSC with Buffer requirements

DEFINITIONS

d-neighborhood

- To model the buffer surrounding the selected core areas we define the d-neighborhood set of a node i as
- For $d \geq 0, d \in \mathbb{N}, i \in V$ the d-neighborhood is defined as the set of sites at distance d from i.

$$\delta_d(i) = \{j \in V_{i \neq j} | \text{the number of jumps from } i \text{ and } j \text{ is max } d\}$$

- $\delta_d^+(i)$ it the d-neighborhood that contains also i.

> $$j \in \delta_d(i) \iff i \in \delta_d(j)$$

GRSC-B: GRSC with Buffer requirements

CONSTRAINTS

d-BUFF.1

Every core site is surrounded by a buffer area of normal sites in its d-neighborhood

$$z_i \leq x_j, \quad \forall j \in \delta_d(i), \quad \forall i \in V$$

d-BUFF.2

Every normal site must have at least one core site in the d-neighborhood

$$x_i \leq \sum_{j \in \delta_d(i)} z_j \quad \forall i \in V$$

GRSC-B: GRSC with Buffer requirements

GRSC-B MODEL:

$$GRSC\text{-}B : \min\{\gamma(u, x, z) |$$
$$(S1\text{-}SQ), (S2\text{-}SQ), (S1\text{-}PROTECT), (S2\text{-}PROTECT), (LINK)$$
$$(d\text{-}BUFF.1), (d\text{-}BUFF.2)\}$$

GRSC-C: GRSC with Connectivity requirements

DEFINITIONS

r-arc-node-separator

- A root r is added to the graph: we can consider $G_r = (V_r, E_r)$ where $V_r = V \cup \{r\}$ and $E_r = E \cup \{(r, i) | i \in V\}$ (so all the nodes from the original graph are connected to r)
- The arcs that connect the nodes to the root r are called **r-arcs** and the set of r-arcs is called A_r
- Given $l \in V$, an **r-arc-node-separator** is a tuple $W = (W_V, W_A)$, where $W_V \subseteq V$ and $W_A \subseteq A_r$, such that if W is removed from G_r then the site l can't be reached by r
- W_l is the set of all possible r-arc-node-separators w.r.t l
- to have connectivity, we want in the solution to have a path from r to all nodes present in the solution

GRSC-C: GRSC with Connectivity requirements

VARIABLES

- Let $y_i, 1 \leq i \leq |V|$ be an auxiliary variable

$$y_i = \begin{cases} 1, & \text{if the land site } i \in V \text{ is connected to } r \text{ via an arc } (r, i) \\ 0, & \text{otherwise} \end{cases}$$

GRSC-C: GRSC with Connectivity requirements

CONSTRAINTS

Let k be the max number of connected components in the reserve.

NCOMP

There's a max of k connected components in the reserve.

$$\sum_{j \in V} y_j \leq k$$

GRSC-C: GRSC with Connectivity requirements

CONSTRAINTS

ALLCON

If a site i is connected to the root, then it must be considered in the reserve area.

$$\sum_{i \in W_V} x_i + \sum_{j \in W_A} y_j \geq x_l, \quad \forall W \in W_l, \quad \forall l \in V$$

YX

If the land site l is in the reserve area of the solution, then for all the $W \in W_l$, I must have at least one node from W_V or one arc from W_A.

$$y_i \leq x_i \quad \forall i \in V$$

GRSC-C: GRSC with Connectivity requirements

GRSC-C MODEL:

$$GRSC\text{-}C : \min\{\gamma(u,x,z)| \\ (S1\text{-}SQ),(S2\text{-}SQ),(S1\text{-}PROTECT),(S2\text{-}PROTECT),(LINK) \\ (ALLCON),(YX),(NCOMP)\}$$

GRSC-CB: GRSC with Buffer and Connectivity

CONSTRAINTS

YZ (replaces YX)

If a site i is connected to the root, then it must be considered in the **core** area.

$$y_i \leq z_i \quad \forall i \in V$$

CORECON (replaces ALLCON)

If the land site l is in the **core** area of the solution, then for all the $W \in W_l$, I must have at least one node from W_V or one arc from W_A.

$$\sum_{i \in W_V} z_i + \sum_{j \in W_A} y_j \geq z_l, \quad \forall W \in W_l, \quad \forall l \in V$$

GRSC-CB: GRSC with Buffer and Connectivity

GRSC-B MODEL:

$$
\begin{aligned}
&GRSC\text{-}CB : \min\{\gamma(u,x,z)| \\
&(S1\text{-}SQ),(S2\text{-}SQ),(S1\text{-}PROTECT),(S2\text{-}PROTECT),(LINK) \\
&(d\text{-}BUFF.1),(d\text{-}BUFF.2) \\
&(CORECON),(YZ),(NCOMP) \\
&(u,x,z,y) \in \{0,1\}^{|S|+3|V|}\}
\end{aligned}
$$

Branch-and-cut framework

Branch-and-cut framework

- In the GRSC-CB and GRSC-C, the family of constraints caused by **connectivity cuts** (CORECON and ALLCON) are exponential in number
- Therefore, a **branch-and-cut** framework is built to separate the connectivity cuts
- To improve the quality of lower bounds, additional valid inequalities are defined and separated:
  - **Species Cuts (SC)**
  - **Cover Inequalities (COVER)**
  - **Species-Cover Cuts (SCC)**
- Only CORECON is described, since ALLCON is symmetrical

Branch-and-cut framework

VALID INEQUALITIES

Species cuts

- A sink node s is defined
- For a certain specie s, every node in V_s is connected with a new arc to the sink node
- r-arc nodes separators are considered with respect to s

$$\sum_{i \in W_V} z_i + \sum_{j \in W_A} y_j \geq u_s, \quad \forall W \in W_s, s \in S_1$$

Branch-and-cut framework

VALID INEQUALITIES

Cover inequalities

- Let $W_s = \sum_{i \in V} w_i^s$
- **Cover** = a set $C_s \subset V_s$ such that $\sum_{i \in C_s} w_i^s \geq W_s - \lambda_s$
- Meaning $V_s \setminus C_s$ is not enough to satisfy the sustainability quota for that specie

$$\sum_{i \in C_s} z_i \geq u_s, \text{if } s \in S_1$$

$$\sum_{i \in C_s} x_i \geq u_s, \text{if } s \in S_2$$

Branch-and-cut framework

VALID INEQUALITIES

Species cover cuts

- Formulated as SC, but the sink is connected to C_s instead of V_s

$$\sum_{i \in W_V} z_i + \sum_{j \in W_A} y_j \geq u_s, \quad \forall W \in W_s, s \in S_1$$

Branch-and-cut framework

CONSTRAINT SEPARATION

CORECON (fractional) separation

- Let $\rho = (\tilde{u}, \tilde{x}, \tilde{z}, \tilde{y})$ is a solution of the LP relaxation at the current node of the branch-and-bound tree
- **Node splitting** is used to create a digraph: every node i is split into two copies:
  - i_1 (entry)
  - i_2 (exit)
- The arcs and their capacities are defined as follows:

Arc	Capacity
$i_1 \to i_2$	$\tilde{z}_i$
$r \to i_1$	$\tilde{y}_i$
$i_2 \to j_1$	∞

- The arc with ∞ capacity forces the flow to pass through the internal node arcs (with capacity $\tilde{z}_i$)

Branch-and-cut framework

CONTRAINT SEPARATION

CORECON (fractional) separation

- A violated connectivity cut $(\overline{W}_V, \overline{W}_A)$ is then identified as the min-cut of the digraph from r to ℓ :

$$\overline{W}_V = \{i \mid (i_1, i_2) \in \text{cut arcs}\}$$
$$\overline{W}_A = \{(r, i) \mid (r, i_1) \in \text{cut arcs}\}$$
$$\sum_{i \in \overline{W}_V} \tilde{z}_i + \sum_{j \in \overline{W}_A} \tilde{y}_j < \tilde{z}_l$$

- If such a cut exists, the corresponding CORECON inequality is violated and can be added to the LP

Branch-and-cut framework

CONTRAINT SEPARATION

CORECON (Integer) separation

- H = connected component of the core induced by $\tilde{z}_i = 1$ that does **not** contain any root arc (i.e., $\tilde{y}_i = 0$ for all $i \in H$) → the component is disconnected from r
- The connectivity cut is defined as (W_V, W_A) such that:

$$W_A = H$$
$$W_V = \{j \mid \{i, j\} \in E,\ i \in H,\ j \notin H\}$$

Branch-and-cut framework

CONTRAINT SEPARATION

CORECON separation

- In both the fractional and integer case, **downlifting** is done
- The CORECON not all cuts are considered, but just the ones
- Not all $j \in W_A$ are considered, but only $j \leq \ell$
- So all connected components are rooted to the node with smallest index

Branch-and-cut framework

CONTRAINT SEPARATION

SCC separation

- Similar process to CORECON
- Difference: applied to the cover set C_s (connected to a sink node) instead of a single node ℓ

COVER separation

- The cuts are the solutions of a knapsack-problem:

$$\min \left\{ \sum_{j \in V_s} \tilde{z}_j q_j \;\middle|\; \sum_{j \in V_s} w_j^s q_j \geq W_s - \lambda_s,\ \mathbf{q} \in \{0,1\}^{|V_s|} \right\}$$

- To solve it, a heuristic is followed:
  - sort the nodes in non-decreasing order by $\tilde{z}_j / w_j^s$
  - construct a cover by iteratively picking nodes in that order (smallest ratio first), until: $\sum_{j \in C_s} w_j^s \geq W_s - \lambda_s$

Branch-and-cut framework

CONTRAINT SEPARATION

Implementation of the cut-loop

1. Separate COVER and SCC/SC (at most 20 times)
2. Separate CORECON

> *For **integer solutions**, only connectivity cuts (CORECON) are separated (Step 2)*

Heuristics

Heuristics

CONSTRUCTION HEURISTIC

- The construction heuristic generates a starting solution for initializing the branch-and-cut
  - **Phase 1**: creates a feasible solution in a greedy fashion
  - **Phase 2**: runs a post-processing to remove unnecessary nodes from S_z

Heuristics

CONSTRUCTION HEURISTIC

Node-cost function

- Let $N(i) = \delta_d^+(i) \setminus S_x$ be the **new nodes** added to the reserve by selecting i.
- **Incremental cost** — cost of the new nodes:

$$C_i = \sum_{j \in N(i)} c_j$$

- **Suitability gain** — how much i helps unprotected species:

$$\mathcal{W}_s(i) = \begin{cases} w_i^s + W_s - \lambda_s & s \in S_1 \\ \sum_{j \in N(i)} w_j^s + W_s - \lambda_s & s \in S_2 \end{cases} \quad (\text{zero if } s \text{ already protected})$$

- **Node-cost function** (lower = better):

$$\Delta_i = \frac{C_i}{\text{gain for } S_1 \cdot \mathbf{1}[\text{S1 unmet}] + \text{gain for } S_2 \cdot \mathbf{1}[\text{S2 unmet}]}$$

Heuristics

CONSTRUCTION HEURISTIC

Phase 1

- In this heuristic the partial solution S is stored as (S_z, S_x) where S_z contains the core nodes and S_x contains all nodes.
- We also keep track of the habitat score of the nodes in the partial solution which will be stored in $W_s(S)$.
- **Goal:** build a feasible solution greedily, growing the core until (PROTECT) is satisfied.
- Let $T(S)$ = set of **helpful** nodes not yet in S_z:
  a node is helpful if adding it to the core could protect at least one currently unprotected species.

Heuristics

CONSTRUCTION HEURISTIC

Phase 1

1. Pick k random nodes as roots (one per component); add them to S_z, expand S_x with their buffer
2. **While** (PROTECT) not satisfied:
   - Find the shortest path (weighted by Δ_i) from S_z to any node in $T(S)$
   - Add all nodes on the path to the core; expand S_x with their buffers
   - Update $T(S)$

> *The computation of the shortest-path is done with the Dijkstra algorithm using the node-cost function defined before*

Heuristics

CONSTRUCTION HEURISTIC

Phase 2

- We iterate through the nodes $i \in S_z$ and check if, after removing i, the solution remains feasible.
- Together with i we remove also the nodes from $delta_d(i)$ which become redundant after removing i, i.e. we remove the set

$$S_x^i = \delta_d^+(i) \setminus \bigcup_{j \in S_z, j \neq i} \delta_d^+(j)$$

- Let i^* be the node whose removal results in the largest improvement in the objective function, remove i^* from S_z and S_x^i from S_x and repeat the process until no additional node can be removed

Heuristics

PRIMAL HEURISTIC

- Incorporated in the branch-and-cut framework (as a callback)

Phase 1

- Essentially the same as the construction heuristic, with 2 differences:
  - In the **node-cost** function: c_i is replaced by $c_i(1 - \tilde{x}_i)$ → to account for nodes already/partially selected in the LP solution
  - The randomly generated starting solutions use nodes with $\tilde{y}_i \geq 0.001$ as seeds

Phase 1

- The same as the construction heuristic

Heuristics

LOCAL-BRANCHING HEURISTIC

- The solution found by the construction heuristic is further improved using the local branching:
  - In each local search iteration, we extend the basic ILP-formulation of the problem through an additional local branching constraint which specifies the r-neighborhood w.r.t. S
  - We impose a time limit, if that is reached this means that no better solution is found in the neighborhood so we increas its size by Δ_r
  - Whenever a best solution is found, the size of the neighborhood is reset to r.

Whenever a best solution is found, the size of the neighborhood is reset to r.

Heuristics

LOCAL-BRANCHING HEURISTIC

Results

TEST

- Given
  - n number of land parcels
  - m number of species
  - k number of max connected areas
- We generate a random instance graph where each node is connected to its neighboors using Delunaay triangulation
- The habitat suitability score function is zero:
  - For the external nodes and s in S_1
  - with probability 20% if the species s is in S_1
  - with probability 10% otherwise
- The suitability quota is defined as: $\lambda_s = \lceil 0.05 \rceil \sum_{i \in V_S} w_i^S$

Results

SCALABILITY